

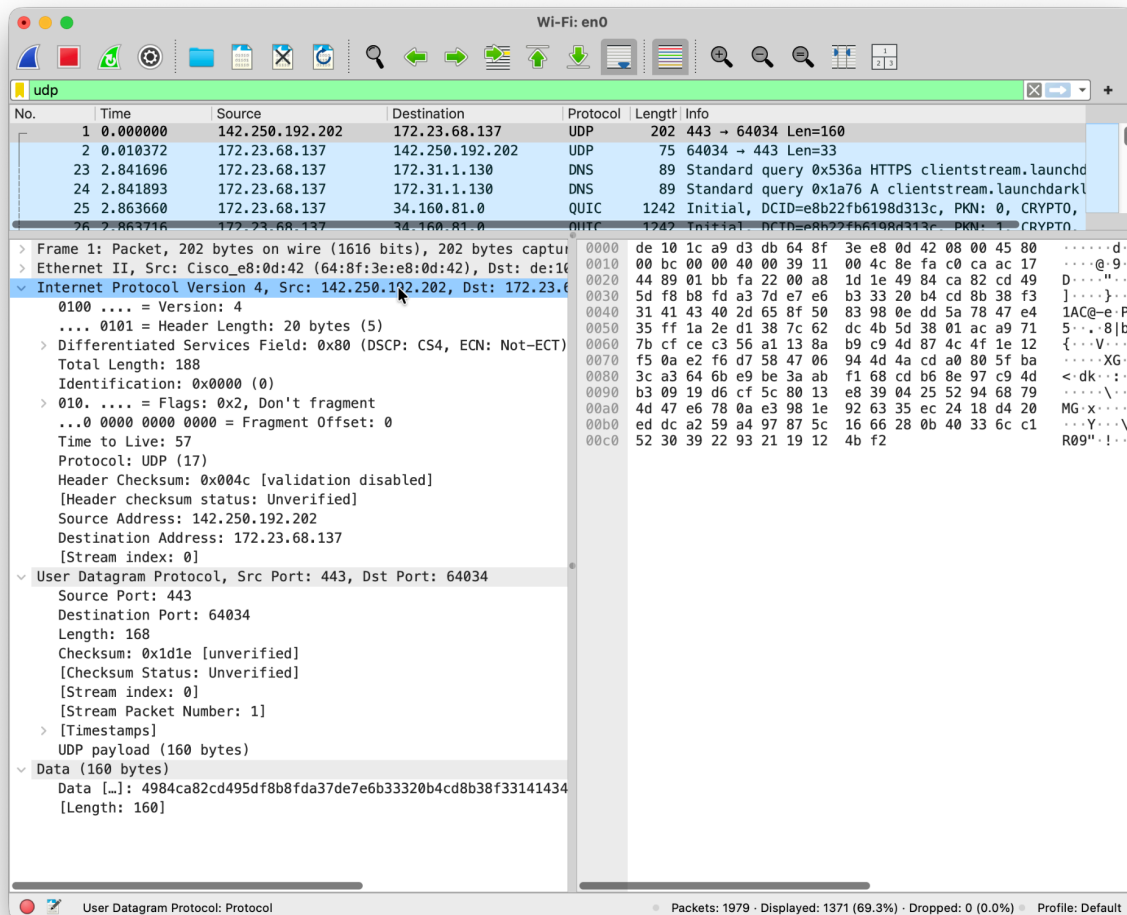
EE673 Assignment 2

Aditya Gautam

220064

Q1: Wireshark Lab: UDP

Course: EE673 Digital Communication Networks



I used the information contained in the bottom left pane of Wireshark to answer all the questions.

1. Select one UDP packet from your trace. From this packet, determine how many fields there are in the UDP header. Name these fields.

By expanding the User Datagram Protocol section in the Wireshark packet details pane, it is evident that the UDP header contains 4 fields.

The names of these fields are:

1. Source Port
2. Destination Port
3. Length
4. Checksum

2. By consulting the displayed information in Wireshark's packet content field for this packet, determine the length (in bytes) of each of the UDP header fields.

The total length of the UDP header is always 8 bytes. Since there are 4 fields in the header, each of the UDP header fields is exactly **2 bytes** (or 16 bits) long.

3. The value in the Length field is the length of what? Verify your claim with your captured UDP packet.

The Length field represents the total length of the entire UDP segment, which includes both the 8-byte UDP header and the encapsulated UDP data payload.

Verification from captured packet: In the captured packet, the Length field shows a value of 168. Looking at the data section directly below the header, the UDP payload size is 160 bytes. Adding the fixed 8-byte UDP header to the 160-byte payload equals exactly 168 bytes, verifying the claim.

4. What is the maximum number of bytes that can be included in a UDP payload?

Since the Length field is 2 bytes (16 bits) long, the maximum decimal value it can hold is $2^{16} - 1$, which equals 65,535 bytes (this is the max size for the entire segment). Because the UDP header takes up a strict 8 bytes, the maximum number of bytes that can be included in the payload alone is $65,535 - 8 = \mathbf{65,527 \text{ bytes}}$.

5. What is the largest possible source port number?

The Source Port field is also 2 bytes (16 bits) long. Therefore, the largest possible source port

number is $2^{16} - 1$, which equals 65,535.

6. What is the protocol number for UDP? Give your answer in both hexadecimal and decimal notation.

By expanding the encapsulating Internet Protocol Version 4 (IPv4) header in Wireshark and looking at the "Protocol" field, the protocol number for UDP is:

- **Decimal:** 17
- **Hexadecimal:** 0x11

7. Examine a pair of UDP packets in which your host sends the first UDP packet and the second UDP packet is a reply. Describe the relationship between the port numbers in the two packets.

When analyzing a request/reply pair of UDP packets, the port numbers are perfectly inverted between the two packets.

- In the first packet (the request), the port numbers were Source Port: 443, Destination Port: 64034.
- In the second packet (the reply), the port numbers flipped to Source Port: 64034, Destination Port: 443.

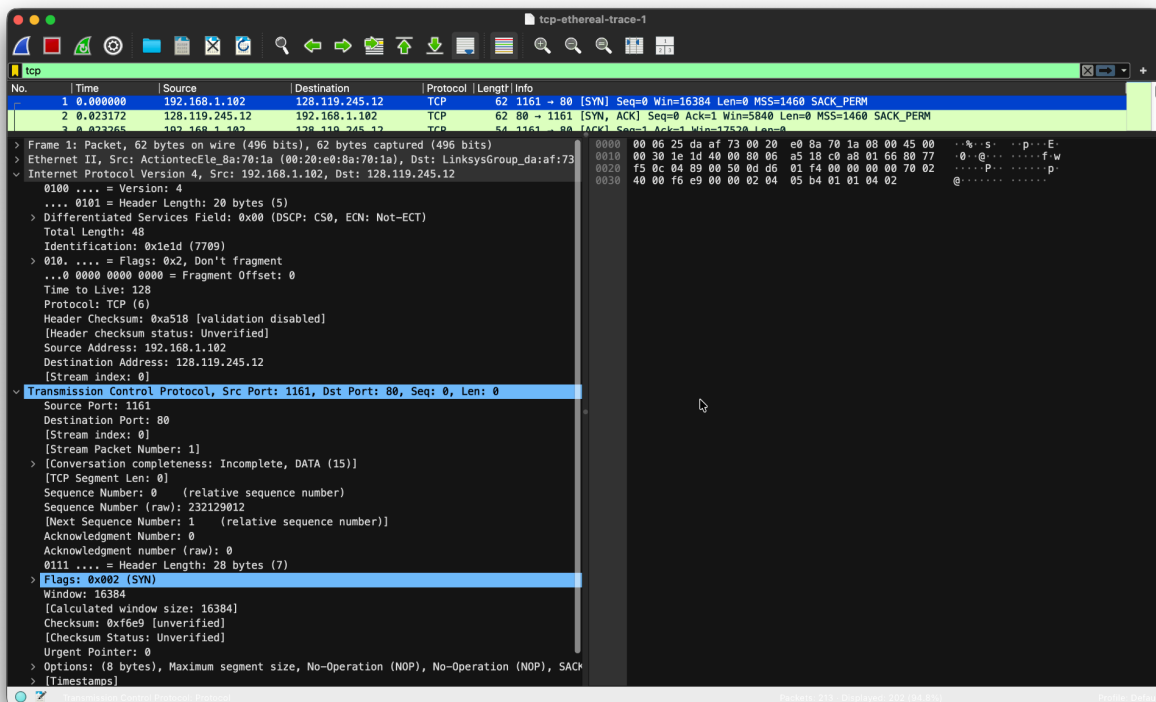
The source port of the sender becomes the destination port for the receiver's reply, and the destination port of the sender becomes the source port for the receiver's reply.

Q1: Wireshark Lab: TCP

(Note that I have used the option provided in the assignment of using the trace file example instead of using my own data, due to huge amount of noise created by various background processes running on my mac system)

1. What is the IP address and TCP port number used by the client computer (source) that is transferring the file to `gaia.cs.umass.edu`?

- Client IP Address: 192.168.1.102
- Client TCP Port: 1161



2. What is the IP address of `gaia.cs.umass.edu`? On what port number is it sending and receiving TCP segments for this connection?

- Server IP Address: 128.119.245.12
- Server TCP Port: 80 (HTTP)

3. What is the IP address and TCP port number used by your client computer (source) to transfer the file to `gaia.cs.umass.edu`?

(Note: This is skipped as we are analyzing the provided trace file).

4. What is the sequence number of the TCP SYN segment that is used to initiate the TCP connection between the client computer and gaia.cs.umass.edu? What is it in the segment that identifies the segment as a SYN segment?

- **Sequence Number (Raw):** 232129012
- **Sequence Number (Relative):** 0
- **Identification:** The segment is identified as a SYN segment because the **Syn** flag is set to 1 (Flags: 0x002) in the TCP header flags field.

```
Sequence Number: 0    (relative sequence number)
Sequence Number (raw): 232129012
[Next Sequence Number: 1    (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment number (raw): 0
0111 .... = Header Length: 28 bytes (7)
> Flags: 0x002 (SYN)
```

5. What is the sequence number of the SYNACK segment sent by gaia.cs.umass.edu to the client computer in reply to the SYN? What is the value of the Acknowledgement field in the SYNACK segment? How did gaia.cs.umass.edu determine that value? What is it in the segment that identifies the segment as a SYNACK segment?

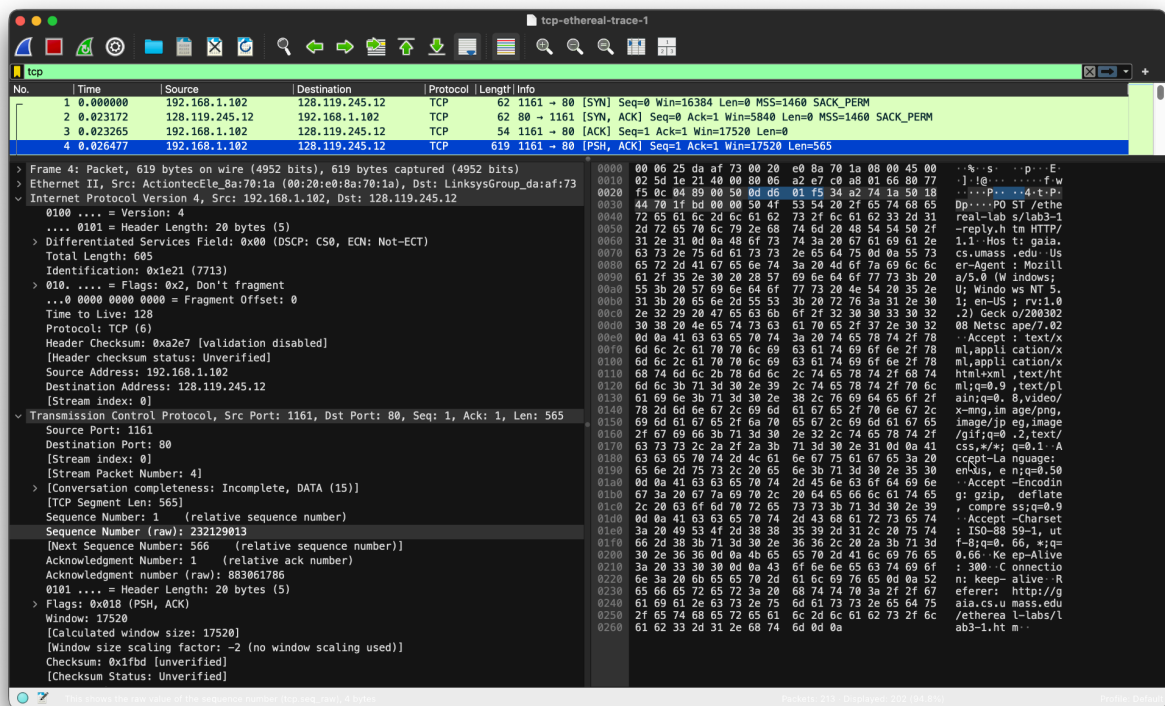
- **Sequence Number (Raw):** 883061785
- **Acknowledgement Field (Raw):** 232129013
- **How it was determined:** The server took the client's initial sequence number (232129012) and added 1 to acknowledge the receipt of the SYN flag.
- **Identification:** It is identified as a SYNACK segment because both the **Syn** and **Ack** flags are set (Flags: 0x012).

```
Sequence Number: 0    (relative sequence number)
Sequence Number (raw): 883061785
[Next Sequence Number: 1    (relative sequence number)]
Acknowledgment Number: 1    (relative ack number)
Acknowledgment number (raw): 232129013
0111 .... = Header Length: 28 bytes (7)
> Flags: 0x012 (SYN, ACK)
```

6. What is the sequence number of the TCP segment containing the HTTP POST command?

This can be observed in Packet 4.

- **Sequence Number (Raw):** 232129013
- **Sequence Number (Relative):** 1



7. Consider the TCP segment containing the HTTP POST as the first segment in the TCP connection. What are the sequence numbers of the first six segments in the TCP connection? At what time was each segment sent? When was the ACK for each segment received? Given the difference between when each TCP segment was sent, and when its acknowledgement was received, what is the RTT value for each of the six segments?

Segment	Sequence Number	Sent Time (s)	ACK Received Time (s)	RTT (s)
1 (POST)	1	0.026477	0.053937	0.027460
2	566	0.041737	0.077294	0.035557
3	2026	0.054026	0.124085	0.070059
4	3486	0.054690	0.169118	0.114428
5	4946	0.077405	0.217299	0.139894
6	6406	0.078157	0.267802	0.189645

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.102	128.119.245.12	TCP	62	1161 → 80 [SYN] Seq=0 Win=16384 Len=0 MSS=1460 SACK_PERM
2	0.023172	128.119.245.12	192.168.1.102	TCP	62	80 → 1161 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 SACK_PERM
3	0.023265	192.168.1.102	128.119.245.12	TCP	54	1161 → 80 [ACK] Seq=1 Ack=1 Win=17520 Len=0
4	0.026477	192.168.1.102	128.119.245.12	TCP	619	1161 → 80 [PSH, ACK] Seq=1 Ack=1 Win=17520 Len=565
5	0.041737	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [PSH, ACK] Seq=566 Ack=1 Win=17520 Len=1460
6	0.053937	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=1 Ack=566 Win=6780 Len=0
7	0.054026	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=2026 Ack=1 Win=17520 Len=1460
8	0.054690	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=3486 Ack=1 Win=17520 Len=1460
9	0.077294	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=1 Ack=2026 Win=8760 Len=0
10	0.077485	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=4946 Ack=1 Win=17520 Len=1460
11	0.078157	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=6406 Ack=1 Win=17520 Len=1460
12	0.124085	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=1 Ack=3486 Win=11680 Len=0
13	0.124185	192.168.1.102	128.119.245.12	TCP	1201	1161 → 80 [PSH, ACK] Seq=7866 Ack=1 Win=17520 Len=1147
14	0.169118	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=1 Ack=4946 Win=14600 Len=0
15	0.217299	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=1 Ack=6406 Win=17520 Len=0
16	0.267802	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=1 Ack=7866 Win=20440 Len=0
17	0.304807	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=1 Ack=9013 Win=23360 Len=0
18	0.305040	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=9013 Ack=1 Win=17520 Len=1460
19	0.305813	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=10473 Ack=1 Win=17520 Len=1460
20	0.306692	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=11933 Ack=1 Win=17520 Len=1460
21	0.307571	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=13393 Ack=1 Win=17520 Len=1460
22	0.308699	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=14853 Ack=1 Win=17520 Len=1460
23	0.309553	192.168.1.102	128.119.245.12	TCP	946	1161 → 80 [PSH, ACK] Seq=16313 Ack=1 Win=17520 Len=892
24	0.356437	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=1 Ack=10473 Win=26280 Len=0
25	0.400164	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=1 Ack=11933 Win=29200 Len=0
26	0.448613	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=1 Ack=13393 Win=32120 Len=0
27	0.500029	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=1 Ack=14853 Win=35040 Len=0
28	0.545052	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=1 Ack=16313 Win=37960 Len=0
29	0.576417	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=1 Ack=17205 Win=37960 Len=0
30	0.576671	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=17205 Ack=1 Win=17520 Len=1460
31	0.577385	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=18665 Ack=1 Win=17520 Len=1460
32	0.578329	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=20125 Ack=1 Win=17520 Len=1460

8. What is the length of each of the first six TCP segments?

- **Segment 1:** 565 bytes (HTTP POST payload)
- **Segment 2:** 1460 bytes (MSS)
- **Segment 3:** 1460 bytes (MSS)
- **Segment 4:** 1460 bytes (MSS)
- **Segment 5:** 1460 bytes (MSS)
- **Segment 6:** 1460 bytes (MSS)

9. What is the minimum amount of available buffer space advertised at the receiver for the entire trace? Does the lack of receiver buffer space ever throttle the sender?

- **Minimum Buffer:** 5840 bytes (Advertised in the initial SYNACK, Packet 2).
- **Throttling:** No. The receiver window grows steadily (reaching over 62,000 bytes) and never drops to zero. The sender is not throttled by the receiver's buffer space.

10. Are there any retransmitted segments in the trace file? What did you check for (in the trace) in order to answer this question?

- **Retransmissions:** No, there are zero retransmitted segments.
- **Method:** I applied the Wireshark display filter `tcp.analysis.retransmission`. The packet list remained empty, indicating no retransmissions occurred.

11. How much data does the receiver typically acknowledge in an ACK? Can you identify cases where the receiver is ACKing every other received segment?

- The receiver typically acknowledges **1460 bytes** of data per ACK (e.g., Ack number increases from 2026 to 3486).
- In this specific trace, the receiver is ACKing every segment individually, rather than every other segment (cumulative ACKs for every single packet received).

12. What is the throughput (bytes transferred per unit time) for the TCP connection? Explain how you calculated this value.

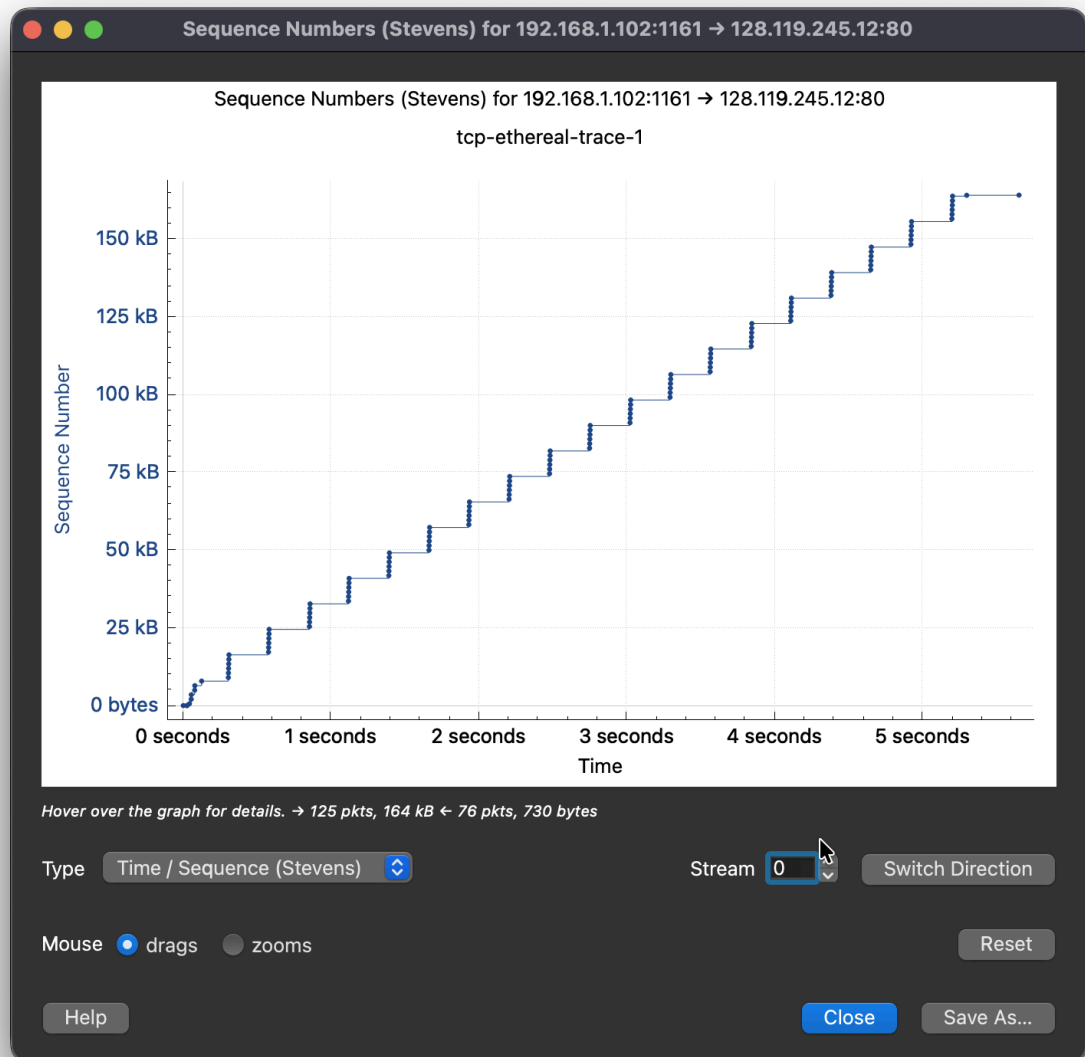
- **Total Data Transferred:** 164,090 bytes (Final Sequence Number 164091 - Initial Sequence Number 1).
- **Total Time:** 5.429 seconds (Last ACK time 5.4558s - First segment time 0.0264s).
- **Throughput Calculation:**

Throughput = 164,090 bytes / 5.429 seconds

This is approximately **30.2 KB/sec**.

13. Use the Time-Sequence-Graph(Stevens) plotting tool to view the sequence number versus time plot of segments being sent from the client to the gaia.cs.umass.edu server. Can you identify where TCP's slowstart phase begins and ends, and where congestion avoidance takes over?

- **Slow Start:** Begins at Time=0. The graph shows an exponential increase in the number of segments sent (steep upward curve) up to approximately 0.2–0.3 seconds.
- **Congestion Avoidance:** Around Time=0.3s, the slope becomes linear. This indicates the transition to Congestion Avoidance, where the window size increases linearly rather than exponentially.

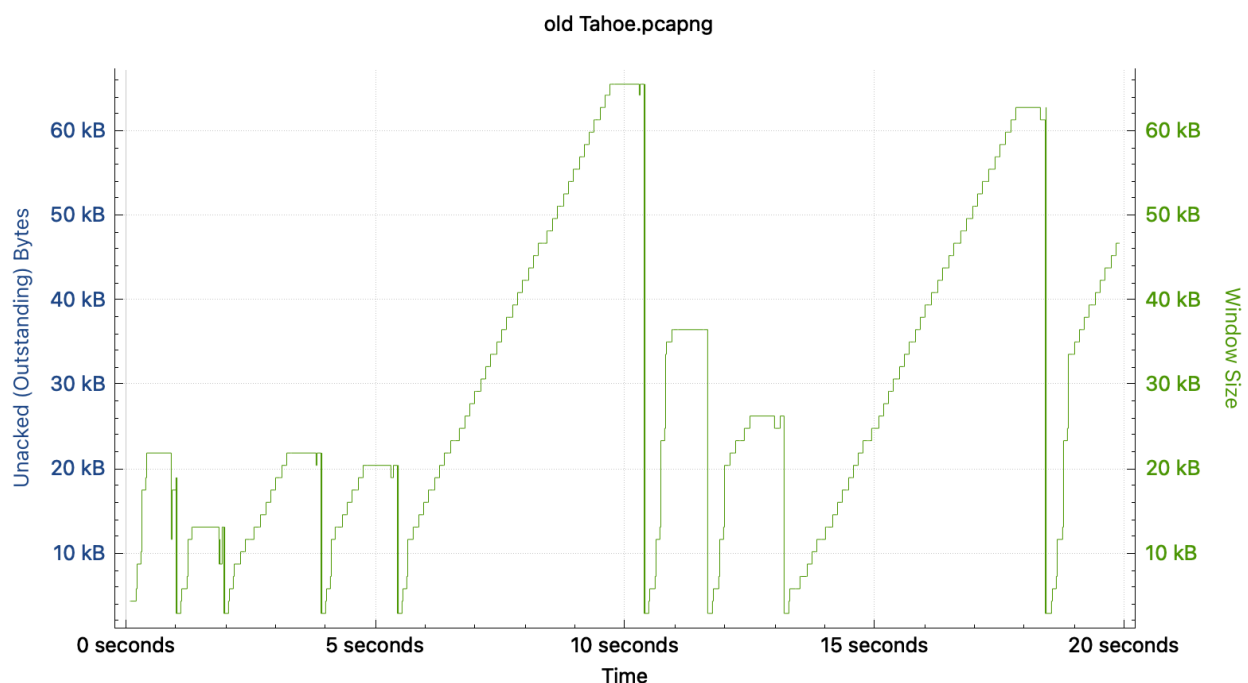


14. Skipped as I followed the option of using the Downloaded Data, as mentioned before.

Q2: Studying TCP Congestion Control in NetSim

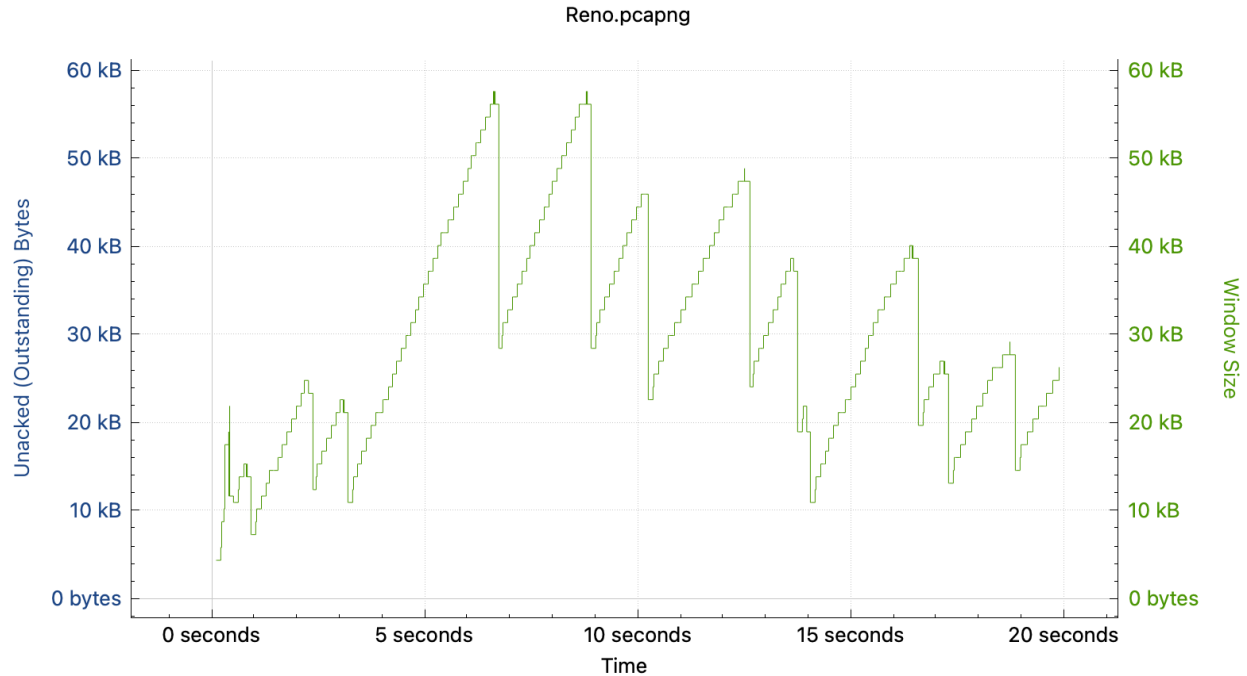
In this simulation, an FTP application was set up to transfer a large file over a lossy 10 Mbps link using TCP. To analyze the behavior of different congestion control mechanisms, the simulation was run three times using Old Tahoe, Reno, and Cubic algorithms. The plots below illustrate the outstanding unacknowledged bytes (Window Size) versus time for each algorithm.

1. Old Tahoe



Discussion: The Old Tahoe plot demonstrates a highly conservative approach to network congestion. As seen in the graph, the window size grows initially (Slow Start and Congestion Avoidance) until packet loss occurs on the lossy link. When a loss is detected, Old Tahoe abruptly drops its congestion window all the way down to a minimum of 1 Maximum Segment Size (MSS) (near 0 bytes on the graph) and restarts the Slow Start phase. This behavior creates a pattern of sharp peaks followed by complete crashes, leaving the link severely underutilized while the window slowly rebuilds from zero.

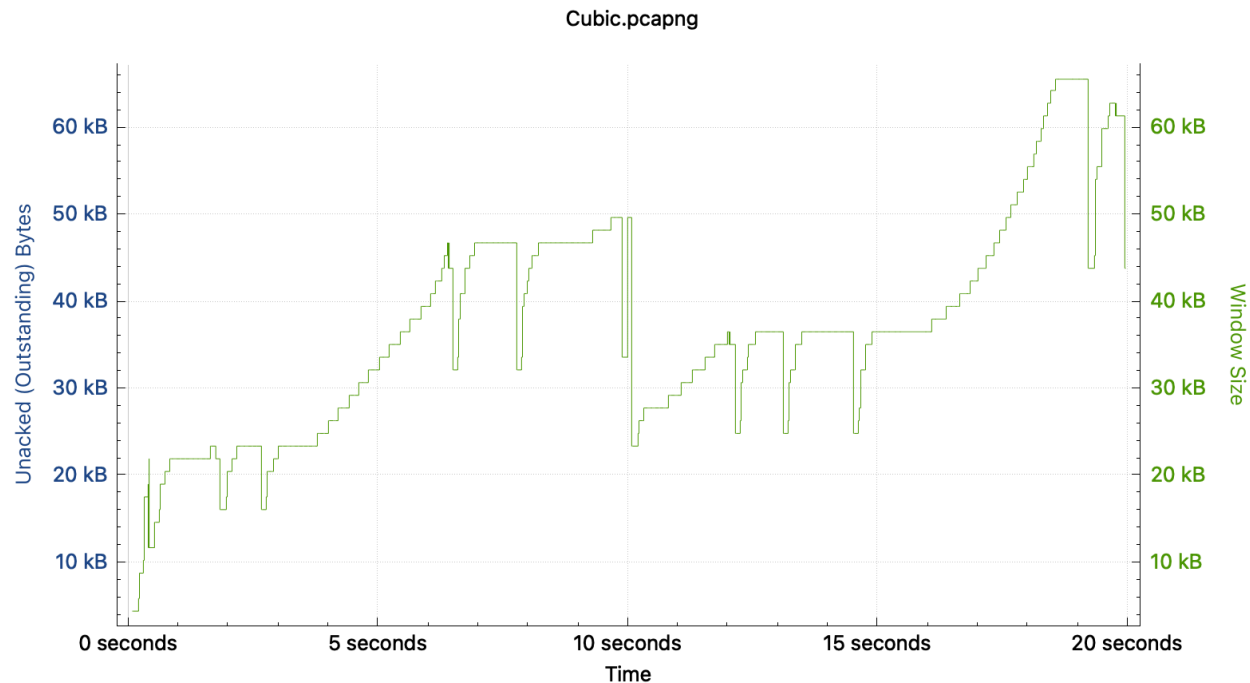
2. TCP Reno



Discussion:

The Reno plot clearly shows the classic TCP "sawtooth" pattern. Unlike Old Tahoe, Reno utilizes Fast Retransmit and Fast Recovery. When packet loss is detected (typically via triple duplicate ACKs), Reno does not reset the window to zero. Instead, it halves the congestion window. You can observe this in the graph where the window size drops from a peak of around 60 kB down to roughly 30 kB, and then resumes linear growth (Congestion Avoidance). By avoiding the drop to zero, Reno maintains a much higher average window size than Old Tahoe, resulting in better throughput.

3. TCP Cubic



Discussion:

The Cubic plot illustrates a more advanced, aggressive approach designed for high-speed networks. Instead of strict linear growth during congestion avoidance, Cubic's window growth is dictated by a cubic function. The graph shows the window size rapidly recovering to the point where the last congestion event occurred, flattening out to probe the network capacity carefully, and then scaling up again. Notice how the window size stays consistently higher (frequently hovering between 30 kB and 60 kB) with fewer drastic drops compared to Reno or Tahoe.

Comparison and Conclusion

When comparing the three plots, the key factor in determining link utilization is the area under the curve, which represents the amount of data consistently kept in flight. Old Tahoe performs the poorest due to its constant drops to zero. Reno improves upon this by only halving its window, but it still takes time to recover linearly. Cubic recovers its window size much faster and spends the majority of its time near the network's maximum capacity.

Which Congestion control algorithm is able to utilize the link capacity to its maximum?

TCP **Cubic** is able to utilize the link capacity to its maximum. By using a cubic growth function, it quickly recovers from packet loss and spends more time operating at a high window size, thereby maximizing the throughput over the 10 Mbps lossy link.

Q3: Speed Test using iPerf

1. Objective: The objective of this task was to measure the maximum achievable bandwidth between two systems on the same IP network using the iPerf3 tool.

2. Collaboration & Methodology: For this task, I collaborated with my classmate, **Daksh Dua**. We performed the test while connected to the IITK Wi-Fi network. We did the test while sitting in different blocks in Hall 9.

- **My Role (Client):** I acted as the client using my macOS system (IP: 172.23.68.137).
- **Partner's Role (Server):** Daksh acted as the server using his Windows system (IP: 172.23.64.58).
- **Process:** After Daksh started the server, I initiated the connection using the command `iperf3 -c 172.23.64.58` to measure the throughput from my system to his.

3. Results and Observations

- **Total Data Transferred:** 39.5 MBytes
- **Average Bitrate:** 33.1 Mbits/sec
- **Observation:** The bitrate showed some variance, peaking at 52.4 Mbits/sec. This is expected in a shared wireless environment where bandwidth is shared among multiple users on the same access point.

4. Screenshots

Here are the screenshots of the communications. The first one is server side, and the second one is client side.

```

PS C:\Users\daksh\Downloads\iperf3.1.1_32> .\iperf3.exe -s
-----
Server listening on 5201
-----
Accepted connection from 172.23.68.137, port 55941
[ 5] local 172.23.64.58 port 5201 connected to 172.23.68.137 port 55942
[ ID] Interval      Transfer    Bandwidth
[ 5]  0.00-1.00    sec  4.02 MBytes  33.7 Mbits/sec
[ 5]  1.00-2.00    sec  4.18 MBytes  35.1 Mbits/sec
[ 5]  2.00-3.00    sec  4.87 MBytes  40.8 Mbits/sec
[ 5]  3.00-4.00    sec  2.39 MBytes  20.1 Mbits/sec
[ 5]  4.00-5.01    sec  1.74 MBytes  14.5 Mbits/sec
[ 5]  5.01-6.00    sec  4.03 MBytes  34.2 Mbits/sec
[ 5]  6.00-7.00    sec  3.10 MBytes  26.1 Mbits/sec
[ 5]  7.00-8.00    sec  6.26 MBytes  52.5 Mbits/sec
[ 5]  8.00-9.01    sec  5.77 MBytes  48.1 Mbits/sec
[ 5]  9.01-10.01   sec  3.13 MBytes  26.2 Mbits/sec
[ 5] 10.01-10.01   sec  0.00 Bytes   0.00 bits/sec
-----
[ ID] Interval      Transfer    Bandwidth
[ 5]  0.00-10.01   sec  0.00 Bytes   0.00 bits/sec      sender
[ 5]  0.00-10.01   sec 39.5 MBytes  33.1 Mbits/sec      receiver
-----
Server listening on 5201
-----

```

```

(base) adityagautam@Adityas-MacBook-Pro ~ % iperf3 -c 172.23.64.58
Connecting to host 172.23.64.58, port 5201
[ 5] local 172.23.68.137 port 55942 connected to 172.23.64.58 port 5201
[ ID] Interval      Transfer    Bitrate
[ 5]  0.00-1.01    sec  4.12 MBytes  34.4 Mbits/sec
[ 5]  1.01-2.00    sec  4.12 MBytes  34.6 Mbits/sec
[ 5]  2.00-3.01    sec  4.88 MBytes  40.9 Mbits/sec
[ 5]  3.01-4.00    sec  2.38 MBytes  19.9 Mbits/sec
[ 5]  4.00-5.01    sec  1.75 MBytes  14.7 Mbits/sec
[ 5]  5.01-6.00    sec  4.00 MBytes  33.7 Mbits/sec
[ 5]  6.00-7.00    sec  3.12 MBytes  26.2 Mbits/sec
[ 5]  7.00-8.00    sec  6.25 MBytes  52.4 Mbits/sec
[ 5]  8.00-9.00    sec  5.75 MBytes  48.3 Mbits/sec
[ 5]  9.00-10.00   sec  3.12 MBytes  26.2 Mbits/sec
-----
[ ID] Interval      Transfer    Bitrate
[ 5]  0.00-10.00   sec 39.5 MBytes  33.1 Mbits/sec      sender
[ 5]  0.00-10.00   sec 39.5 MBytes  33.1 Mbits/sec      receiver

iperf Done.
(base) adityagautam@Adityas-MacBook-Pro ~ %

```